

فصل هفتم

VIEW

۷-۱ تعریف دید

یک دید در SQL یک جدول مجازی مبتنی بر نتایج یک عبارت SQL می باشد که به طور منطقی (نه فیزیکی) در پایگاه داده ذخیره می شود. در واقع یک دید شامل سطرها و ستون هایی دقیقاً شبیه یک جدول واقعی می باشد. فیلدهای یک دید فیلدهایی هستند که از یک یا تعداد بیشتری جدول واقعی در پایگاه داده می آیند. شما می توانید توابع SQL، WHERE، و عبارات JOIN را به یک دید اضافه کنید.

دید ها برای اهداف امنیتی در پایگاه داده کاربرد دارد. دیدها کاربر را محدود به دیدن سطرها و ستون های خاصی می کنند، برای این منظور ما می توانیم هنگام ایجاد یک دید محدودیتی در دسترسی به سطرها و ستون های خاصی برای یک کاربر مشخص اعمال کنیم.

۷-۱-۱ مزایای دید

- امنیت: با استفاده از دید ها می توان به هر کاربر اجازه دسترسی به داده های مشخصی را داد، و بنابراین در دسترسی به داده های ذخیره شده در پایگاه داده محدودیت داشته و به این صورت می توان امنیت را افزایش داد.
- ساده سازی پرسش ها: یک دید می تواند داده ها را از چندین جدول مختلف دریافت کند و بصورت یک جدول واحد آن را تحویل دهد. بنابراین با استفاده از دید می توان پرسش از چند جدول را به پرسش از یک جدول تبدیل کرد.
- ساده سازی ساختار: از آن جا که دیدها پایگاه داده را به شکل یک مجموعه از جداول مجازی که مختص همان کاربر است در اختیار کاربر قرار می دهند، دیدها می توانند یک دیدگاه "شخصی" از ساختار پایگاه داده را به کاربر بدهند و به این صورت ساختار پایگاه داده برای هر کاربر ساده می شود.
- سازگاری^{۳۴}: یک دید حتی اگر جداول منبع مهم آن تقسیم شود، دوباره بازسازی شود یا تغییر نام پیدا کند یک تصویر ثابت و غیرمتغیر از ساختار پایگاه داده را برای ما فراهم می کند.

^{۳۴} Consistency

- **جامعیت داده ها^{۳۵}:** اگر داده ها از طریق یک دید وارد شوند، DBMS به صورت خودکار داده ها را بررسی می کند تا مطمئن شود محدودیت های مشخص شده جامعیت داده ها را تأمین میکند.

۷-۲ معایب دید

- **کارایی^{۳۶}:** دیده ها در ظاهر یک جدول واحد هستند، اما وقتی پرسشی داده می شود، DBMS باید از چندین جدول پایه دید نتایج را برگرداند. بنابراین اگر دید بر مبنای دستورات پیچیده SQL و از چندین جدول تشکیل شده باشد، اجرای پرسش روی آن دید ممکن است زمان قابل توجهی را به خود اختصاص دهد.
- **محدودیت بروزرسانی:** هنگامی که یک کاربر می خواهد سطرهای یک دید را بروز رسانی کند، DBMS باید درخواست را روی جدول های پایه ای دید اجرا کند. در صورتیکه دید ها ساده باشد این کار ممکن است، اما دیدهای پیچیده تر اغلب فقط خواندنی هستند و نمی توان آن ها بروز رسانی کرد.

۷-۲ ایجاد یک دید

برای ایجاد یک دید در SQL از قالب کلی زیر استفاده میشود:

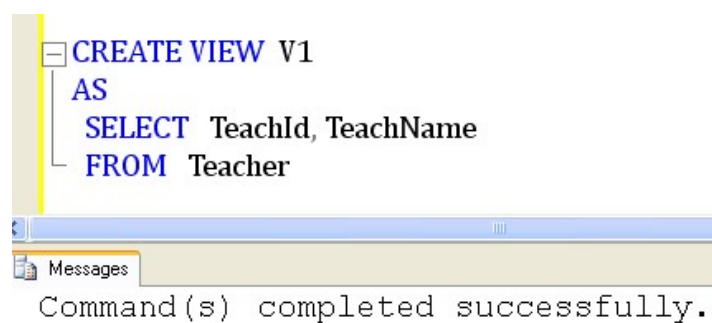
```
CREATE VIEW view_name AS  
SELECT Column_Name(s)  
FROM Table_Name  
WHERE condition
```

نکته: یک دید همیشه داده های بروز شده را نشان می دهد. هر زمانی که یک کاربر دیدی را اجرا می کند، موتور پایگاه داده با استفاده از عبارت SQL ی آن دید، داده ها را دوباره ایجاد می کند.

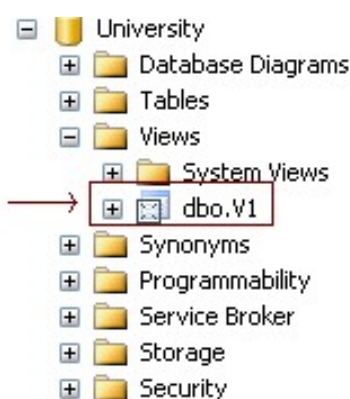
مثال ۷-۱: یک دید روی جدول اساتید از کد استادی و نام استاد ایجاد کنید.

^{۳۵} Data Integrity

^{۳۶} Performance



توضیح مثال: طبق قالب کلی دیدی به نام V1 روی جدول اساتید ایجاد کردیم. اگر در فولدر Views در زیر شاخه پایگاه داده مشاهده کنید یک دید با نام V1 اضافه شده است. حال می توان به کاربری از پایگاه داد فقط اجازه دسترسی به این دید را داد و به او اجازه مشاهده و ویرایش سایر جداول و اشیا پایگاه داده را نداد.



شکل ۱-۷

این دید دارای دو ستون به همان نام های ستونهای متناظر در جدول می باشد. اگر بخواهیم نام ستون های دید را تغییر دهیم (برای مثال کد استادی را Code و نام استاد را Name قرار دهیم) دستور زیر را وارد میکنیم.

CREATE VIEW V2 (Code, Name)

AS

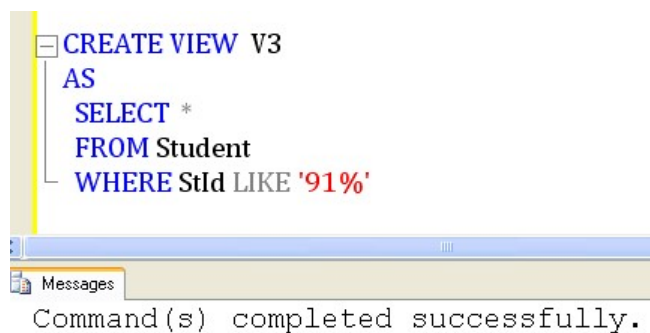
```
SELECT TeachId, TeachName
FROM Teacher
```

حال پس از ایجاد دید همانند یک جدول می توان به آن دسترسی داشت برای مثال دستور SELECT زیر روی دید V1 زده شده است.



	TeachId	TeachName
1	1	مریم حسینی
2	10	محمد اسدی
3	2	علی آقاچانی
4	3	سید علی بیره
5	4	زینب نبوتی
6	5	مریم یزدی
7	6	محمد رضا سهیل د

مثال ۷-۲: دستوری بنویسید که یک دید از اطلاعات تمام دانشجویان ورودی ۹۱ را ایجاد کند.



```

CREATE VIEW V3
AS
SELECT *
FROM Student
WHERE StId LIKE '91%'

```

Command(s) completed successfully.

مثال ۷-۳: از دید V3 اطلاعات دانشجویان رشته معماری را برگردانید.



	StId	Name	Degree	Major	Tel
1	9127887081	محمد رضا شعبانی	کارشناسی	معماری	NULL
2	9128089689	مجید رضازاده	کارشناسی	معماری	NULL
3	9128981231	نیما پاشایی	کارדانی	معماری	NULL

نکته: می توان روی بیش از یک جدول نیز دید ایجاد کرد.

مثال ۷-۴: دیدی با نام V4 ایجاد کنید، تا شماره، نام دانشجویان و اطلاعات انتخاب واحد آنها که در سال تحصیلی ۹۱-۹۲ انتخاب واحد کرده اند را در بر داشته باشد.

```
CREATE VIEW V4
AS
SELECT Student.StId, Name, Major, Grade, Term, [Year]
FROM Student JOIN StCo
ON Student.StId=StCo.StId
WHERE [Year]='91-92'
```

Messages
Command(s) completed successfully.

۷-۳ تغییر یک دید

می توان دیدی را که قبلاً ایجاد شده است را تغییر داد، تغییر دید روی رویه های ذخیره شده^{۳۷} یا تریگرهای^{۳۸} وابسته تأثیری نمی گذارد، همچنین مجوزها را نیز تغییر نمی دهد.

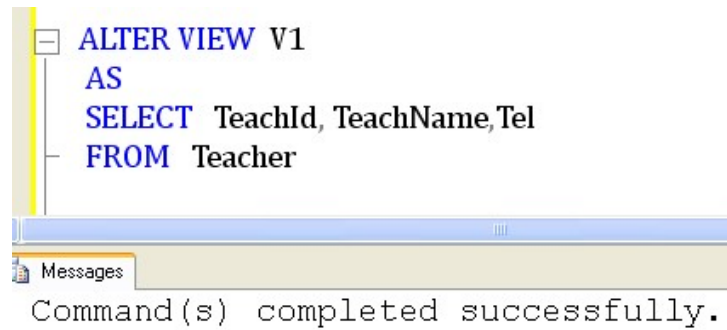
می توان با استفاده از قالب کلی زیر یک دید را تغییر داد:

```
ALTER VIEW view_name [ ( column [ ,...n ] ) ]
[ WITH <view_attribute> [ ,...n ] ]
AS
select_statement
```

مثال ۷-۵: دید V1 را طوری تغییر دهید که اطلاعات تماس استاد را نیز شامل شود.

^{۳۷} Stored procedure

^{۳۸} trigger

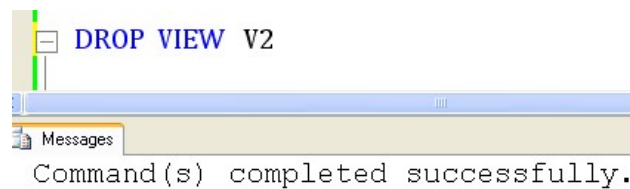


۷-۴ حذف یک دید

برای حذف یک دید که قبلاً ایجاد شده است از دستور زیر استفاده می شود:

```
DROP VIEW view_name
```

مثال ۷-۶: دید V2 را حذف کنید.



۷-۵ تأثیر تغییرات جدول روی دید و بالعکس

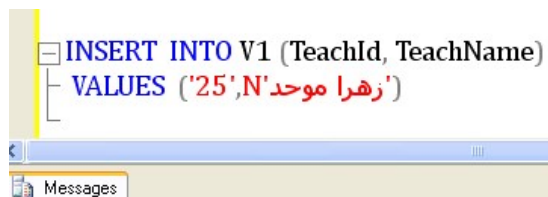
همانگونه که پیش تر گفتیم دید از روی جدول ایجاد میشود و پس از ایجاد آن همانند یک جدول با آن برخورد می شود، بررسی تأثیرات متقابل تغییرات جدول بر روی دید حاصل از آن و بالعکس مسأله مهمی می باشد که در این بخش به آن پرداخته می شود. بهتر است دو حالت را در نظر بگیریم. حالت اول دیدهای حاصل از یک جدول و حالت دیگر دیدهای حاصل از چند جدول میباشد.

در ابتدا حالت اول را بررسی میکنیم، فرض کنید یک دید داریم که حاصل از تنها یک جدول می باشد.

۱- اگر روی جدولی عملیات درج، تغییر و حذف را اعمال کنید، روی دید حاصل از جدول نیز این عملیات اعمال می شود.

۲- اگر بر روی دید عملیات درج، تغییر و حذف را اعمال کنید، بر روی جدول متناظر آن نیز این عملیات اعمال می شود.

مثال ۷-۷ : استادی با کد استادی ۲۵ و نام زهرا موحد را به دید ۷۱ اضافه کنید و نتیجه آن را روی جدول teacher بررسی کنید.



(1 row(s) affected)

پس از اجرای دستور فوق اگر جدول Teacher را مشاهده کنید، می بینید یک سطر با مشخصات فوق به آن اضافه شده است. در ارتباط با تغییر و حذف از دید و تأثیر آن روی جدول و برعکس نیز به همین صورت می توانید نتایج را بررسی کنید.

TeachId	TeachName	Tel
25	زهرا موحد	NULL

حالت دوم هنگامی است که دید شما از بیش از یک جدول تشکیل شده است. در ادامه این حالت را توضیح می دهیم.

۱- اگر روی هر کدام از جداول عملیات درج، تغییر و حذف را اعمال کنید، روی دید حاصل از جداول نیز این عملیات اعمال می شود.

۲- اگر روی دید عمل درج صورت گیرد، در صورتیکه این عمل درج تناقضی با جامعیت داده ها نداشته باشد، بروز رسانی مشکلی ایجاد نمی کند، مثلاً ممکن است در دید شما ستونی که کلید اصلی یکی از جداول است نباشد، در این صورت اگر سطری به دید اضافه شود، باعث ایجاد خطا می شود، چرا که ستون کلید اصلی در جداول نمی تواند مقدار NULL بگیرد.

۳- اگر روی دید بروز رسانی صورت گیرد، این بروز رسانی بر روی جداول مرتبط نیز صورت میگیرد.

۴- حذف سطری از دید باعث ایجاد خطا می شود.

مثال ۷-۸: دید ۷۴ را که در مثال ۷-۴ ایجاد کردیم را در نظر بگیرید، حال فرض کنید سطری به جدول Course اضافه می کنیم، دوباره از دید یک SELECT می گیریم که در نتیجه

مشاهده می کنید که این سطر به دید نیز اضافه می شود.

شکل زیر یک SELECT اولیه از دید ۷۴ می باشد:

SELECT * FROM V4

00 %

Results Messages

	Std	Name	Major	Grade	Term	Year
1	9119822125	علیرضا مالگیر	رایانه	14.00	3	91-92
2	9119822125	علیرضا مالگیر	رایانه	8.00	1	91-92
3	9119822125	علیرضا مالگیر	رایانه	17.00	1	91-92
4	9119822125	علیرضا مالگیر	رایانه	18.00	2	91-92
5	9128089689	مجید رضازاده	معماری	20.00	2	91-92
6	9128089689	مجید رضازاده	معماری	18.50	2	91-92
7	9128089689	مجید رضازاده	معماری	19.00	2	91-92
8	9128089689	مجید رضازاده	معماری	18.00	3	91-92

سپس یک سطر به جدول Course اضافه می کنیم:

INSERT INTO StCo (Std,CoId,Term,Year,Grade,TeachId)
VALUES ('9012769801','ARCH218',1,'91-92',18,3)

00 %

Messages

(1 row(s) affected)

حال مجدد از دید یک SELECT میگیریم:

SELECT * FROM V4

00 %

Results Messages

	Std	Name	Major	Grade	Term	Year
1	9119822125	علیرضا مالگیر	رایانه	14.00	3	91-92
2	9119822125	علیرضا مالگیر	رایانه	8.00	1	91-92
3	9119822125	علیرضا مالگیر	رایانه	17.00	1	91-92
4	9119822125	علیرضا مالگیر	رایانه	18.00	2	91-92
5	9128089689	مجید رضازاده	معماری	20.00	2	91-92
6	9128089689	مجید رضازاده	معماری	18.50	2	91-92
7	9128089689	مجید رضازاده	معماری	19.00	2	91-92

فصل هشتم

توابع و رویه های ذخیره شده

۸- ۱ توابع و رویه ها

تا کنون تمام پرسش هایی که با کد های T-SQL ایجاد کردیم همان لحظه نیز اجرا می کردیم و به صورت واحدی جداگانه ذخیره نکردیم، حال در این فصل به بررسی توابع و رویه های ذخیره شده برای ذخیره سازی دائمی دستورات می پردازیم. در SQL SERVER میتوان دستورالعمل های مورد نیاز را برای اجرا به شکل های مختلف به صورت اشیا مستقل در پایگاه داده ایجاد و ذخیره نمود تا در صورت نیاز امکان اجرای آنها وجود داشته باشد. توابع و رویه ها را یک بار ایجاد می کنید و بار ها و بار ها از آنها استفاده می کنید. به این منظور میتوان از توابع، رویه ها یا Triggerها استفاده نمود. که در ادامه با توابع و رویه ها آشنا خواهید شد.

۸- ۲ توابع:

توابع در SQL SERVER به دو نوع کلی تقسیم می شوند، که عبارتند از توابع سیستمی و توابع تعریف شده توسط کاربر.

نحوه تعریف و استفاده از توابع تعریف شده کاربر یا UDF^{۳۹}

به طور کلی میتوان توابع تعریف شده توسط کاربر در SQL را به گروه های زیر تقسیم نمود:

- توابع اسکالر
- توابع جدولی

توابع اسکالر : توابعی هستند که توانایی اجرای یک یا چند دستورالعمل را دارند و میتوانند نتایج محاسبات و انجام عملیات را بصورت یک مقدار مشخص به محل فراخوانی بازگشت دهند. شکل کلی نحوه تعریف این نوع از توابع به صورت زیر می باشد:

CREATE FUNCTION (...., نوع داده نام پارامتر) نام تابع

RETURNS نوع داده بازگشتی

[AS]

Begin

^{۳۹} User-Defined Function

بدنه دستورات تابع

Return (مقدار بازگشتی)

End

مثال ۸-۱: تابعی بنویسید که طول و عرض مستطیلی را بگیرد و مساحت آن را برگرداند.

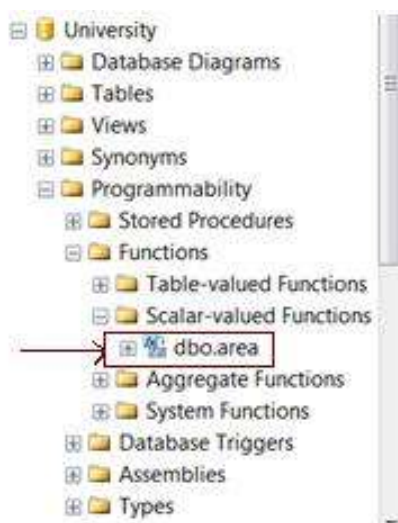
```
CREATE FUNCTION area (@width real , @height real)
RETURNS real

BEGIN
    DECLARE @res AS decimal (8,3)
    SET @res = @width * @height
    RETURN @res
END
```

Messages

Command(s) completed successfully.

توضیح مثال: پس از اجرای تابع فوق اگر در پایگاه داده مربوطه در مسیر Programmability- >Functions->scalar-valued Function مشاهده کنید می توانید نام تابع ایجاد شده را مشاهده نمایید.

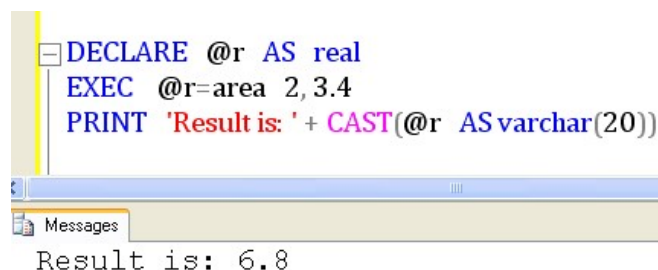


شکل ۸-۱

نکته: به منظور فراخوانی و استفاده از تابع می توان از دستور `exec` استفاده نمود.

`Exec output= Function_name parameters`

مثال ۸-۲: تابع فوق را با طول و عرض ۲ و ۳,۴ اجرا کنید و نتیجه آنرا نمایش دهید.



```

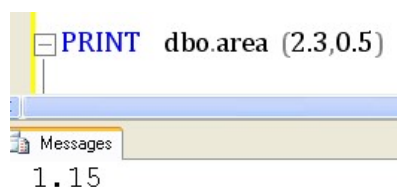
DECLARE @r AS real
EXEC @r=area 2,3.4
PRINT 'Result is: ' + CAST(@r AS varchar(20))

```

Messages

Result is: 6.8

در صورت لزوم میتوان مقدار بازگشتی توابع را با دستور **print** مستقیماً مورد استفاده قرار داد. مثال زیر نمونه ای از این حالت می باشد.



```

PRINT dbo.area (2.3,0.5)

```

Messages

1.15

توضیح مثال: هنگامی که تابع را همراه با دستور **PRINT** اجرا می کنید باید دو نکته را مد نظر بگیرید؛ اولاً در این حالت باید **dbo** را قبل از نام تابع قرار دهید و ثانیاً پارامترهای تابع را داخل پرانتز قرار دهید.

نکته: در هنگام تعریف تابع میتوان برای پارامترها مقدار پیش فرض تعیین نمود. اگر در زمان فراخوانی مقداری ارسال گردد از آن مقدار و در غیر این صورت از مقدار پیش فرض استفاده میشود.

```

Create Function Area_1 (@width real=2 , @height real=3)
Returns real
Begin
Declare @res as decimal (8,3)
Set @res =@width * @height
Return (@res)
end

```

100 %

Messages

Command(s) completed successfully.

حال فراخوانی تابع Area_1 را بصورت زیر داریم:

```

declare @r as real
Exec @r = area_1 default, default
print 'result is:'
print @r

```

100 %

Messages

result is:
6

توضیح مثال: همانطور که مشاهده می کنید به جای مقدار دو پارامتر default نوشتیم در این صورت بجای هر یک از پارامترها مقدار پیش فرض یعنی ۲ و ۳ را قرار می دهد. می توان به جای هر یک از default ها مقدار پارامتر دلخواه نیز گذاشت که در این صورت با آن مقادیر تابع اجرا می شود.

مثال ۸-۳: تابعی بنویسید که تعداد دروسی که دانشجوی دلخواهی برداشته است را برگرداند، سپس تابع را برای دانشجویی با شماره دانشجویی ۸۸۱۶۷۸۲۳۲۳ اجرا کنید.

```

CREATE FUNCTION Num_Course(@Id AS VARCHAR(10))
RETURNS int
BEGIN
    DECLARE @C AS int
    SELECT @C = COUNT(CoId)
    FROM StCo
    WHERE StId =@Id
RETURN @C
END

```

Messages

Command(s) completed successfully.

سپس تابع را برای دانشجوی ۸۸۱۶۷۸۲۳۲۳ به صورت زیر اجرا می کنیم:

```

print 'The Number Of Courses Is: '
print dbo.Num_Course ('8816782323')

```

100 %

Messages

The Number Of Courses Is:
3

نکته: با استفاده از گزینه SCHEMABINDING می توان تابع را به گونه ای تعریف نمود تا امکان تغییر یا حذف اشیایی که بوسیله تابع مورد استفاده قرار میگیرند وجود نداشته باشد.

نکته:هم چنین با استفاده از گزینه ENCRYPTION می توان تابع را بگونه ای تعریف نمود تا امکان مشاهده و دسترسی به محتویات تابع امکانپذیر نباشد.

با این کار مثلاً اگر کسی(یا خودتان)خواستید جدولی (یا هر شی ای) را تغییر دهید که اجرای تابع را با مشکل مواجه می کند، با این پارامتر مطمئن باشید که مشکلی در اجرای تابع نخواهید داشت.

مثال ۸-۴: تابعی بنویسید که متوسط فروش کارمندی دلخواه را برگرداند. (به همراه SCHEMABINDING, ENCRYPTION)

```

CREATE FUNCTION averageSale (@EmployeeId AS char(10))
RETURNS int
WITH ENCRYPTION, SCHEMABINDING
AS
BEGIN
    DECLARE @sale AS int
    SELECT @sale = AVG(TotalPrice) FROM dbo.[Order]
    WHERE (EmployeeId=@EmployeeId)
    RETURN @sale
END

```

Messages

Command(s) completed successfully.

توضیح مثال: پس از ایجاد تابع چون از SCHEMABINDING استفاده کرده اید اگر سعی کنید جدول Order را حذف کنید با خطا مواجه می شوید، هم چنین به دلیل استفاده از ENCRYPTION نمی توانید به محتوای تابع دسترسی داشته باشید.

نکته: بمنظور مشاهده محتویات تابعی که بدون گزینه SCHEMABINDING ایجاد شده است میتوان از رویه سیستمی SP_helptext استفاده کرد.

مثال ۸-۵: با استفاده از رویه SP_helptext محتویات تابع area را مشاهده کنید.

```

exec sp_helptext area

```

100 %

Results Messages

	Text
1	Create Function area (@width real , @height real)
2	Returns real
3	Begin
4	Declare @res as decimal (8,3)
5	Set @res =@width * @height
6	Return (@res)
7	end

۸- ۳ رویه^{۴۰} ها:

رویه ها نیز همانند توابع توانایی اجرای دستورالعمل ها را دارند. رویه ها اشیایی هستند که در اولین بار کامپایل شده و فرمت آن ذخیره می شود (کد کامپایل شده) و در دفعات بعدی می توان آن را فراخوانی کرد و این مورد خود باعث افزایش سرعت اجرای آن می شود. اما Function ها در هر بار فراخوانی آنها، ابتدا کامپایل و سپس اجرا میشوند. در ادامه با تفاوت های بیشتر آشنا می شوید:

۱. توابع حتما باید مقداری را به عنوان خروجی برگردانند در صورتی که در رویه ها اختیاری می باشد (رویه می تواند صفر یا چند مقدار برگرداند).
۲. توابع فقط می توانند پارامترهای ورودی داشته باشند در صورتی که رویه ها می توانند هم پارامترهای ورودی و هم پارامترهای خروجی داشته باشند.
۳. توابع می توانند از داخل یک رویه فراخوانی شوند در حالی که نمی توان در داخل یک تابع اقدام به فراخوانی یک رویه کرد.
۴. رویه ها نمی توانند در داخل دستورات SELECT استفاده شوند درحالیکه تابع می تواند در داخل دستورات SELECT جاسازی شوند.
۵. داخل رویه ها می توان از دستور SELECT, DELETE, UPDATE و INSERT استفاده کرد، اما داخل توابع فقط اجازه استفاده از SELECT را داریم.
۶. رویه ها نمی توانند در هر جای دستورات SQL استفاده شوند، برای نمونه نمی توانند در قسمت های WHERE/HAVING, SELECT استفاده شوند درحالیکه این محدودیت برای تابع وجود ندارد.
۷. مهمترین ویژگی رویه ها نسبت به تابع ها نگهداری و قابلیت استفاده مجدد (execution plan) می باشد درحالیکه برای تابع ها هر بار تابع کامپایل می شود.

رویه ها به طور کلی به دو گروه عمده تقسیم میشوند، که عبارتند از رویه های سیستمی و رویه های ذخیره شده توسط کاربر.

^{۴۰} procedure

رویه سیستمی

رویه های سیستمی رویه هایی هستند که توسط SQL ایجاد و مدیریت می شوند، و در صورت نیاز می توان از آنها استفاده نمود. رویه های سیستمی SQL را می توان در پایگاه داده master در شاخه programmability شاخه System Stored Procedure مشاهده نمود. نام این رویه ها غالباً با SP_ آغاز می شوند. یکی از این رویه های سیستمی که پیشتر با آن آشنا شدیم رویه SP_helptext بود که برای نمایش بدنه تابع بکار برده می شود. علاوه بر این رویه ها، هر پایگاه داده نیز رویه های سیستمی مخصوص به خود را نیز می تواند در اختیار داشته باشد که در سطح پایگاه داده تعریف می گردد. بعنوان یک مثال تابع SP_who نشان می دهد که چه Login ها و User هایی در سیستم تعریف شده است.

```
execute sp_who
```

spid	ecid	status	loginame	hostname	blk	dbname	cmd	req
38	0	sleeping	sa		0	master	TASK MANAGER	0
39	0	sleeping	sa		0	master	TASK MANAGER	0
40	0	sleeping	sa		0	master	TASK MANAGER	0
41	0	sleeping	sa		0	master	TASK MANAGER	0
51	0	sleeping	SEPIDEH-PC\...	SEPIDEH-PC	0	master	AWAITING COMMA...	0
52	0	runnable	SEPIDEH-PC\...	SEPIDEH-PC	0	University	SELECT	0
53	0	sleeping	SEPIDEH-PC\...	SEPIDEH-PC	0	University	AWAITING COMMA...	0

رویه های ذخیره شده توسط کاربر

همانند توابع تعریف شده توسط کاربر این رویه ها را نیز کاربر تعریف می کند. شکل کلی نحوه استفاده از این نوع رویه ها به صورت زیر می باشد:

(....., نوع داده نام پارامتر) نام رویه

AS

[BEGIN]

بدنه دستورات رویه

[END]

رویه ها برعکس توابع به طور مستقل می توانند اجرا شوند. و همزمان می توانند چند مقدار را برگردانند. برای مکانیزه کردن عملیات حتی تابع را می توان در آن فراخوانی کرد.

مثال ۸-۶: رویه ای بنویسید که مشخصات درسی با کد درسی مشخص را برگرداند. سپس این رویه را برای درسی با کد ARCH119 اجرا کنید.

```
CREATE PROCEDURE Course_Info (@CourseId As varchar(10))
AS BEGIN
SELECT * FROM Course WHERE CoId= @CourseId
END
```

Messages
Command(s) completed successfully.

```
EXECUTE Course_Info 'ARCH119'
```

100 %

Results Messages

	COID	COTITLE	CREDIT	COTYPE
1	ARCH119	معماری معاصر	2	نظری-عملی

به منظور فراخوانی رویه ها میتوان از دستور EXECUTE استفاده نمود:

پارامترها نام رویه EXECUTE

نکته: همانند توابع در رویه ها نیز برای آرگومان ها میتوان مقدار NULL یا هر مقدار دیگری را به عنوان مقدار پیش فرض اعلام نمود.

مثال ۸-۷: رویه ی زیر مثالی از مقدار پیش فرض پارامتر با NULL می باشد.

```

CREATE PROCEDURE listOfEmployee (@lastname nvarchar (30)=NULL)
AS
BEGIN
    IF ( @lastname IS NULL)
        SET @lastname=N'کبیری'
    SELECT FirstName, LastName
    FROM Employee
    WHERE Lastname =@lastname
END

```

Messages

Command(s) completed successfully.

EXECUTE listOfEmployee

EXECUTE listOfEmployee N'علیزاده'

	FirstName	LastName
1	علی	کبیری

	FirstName	LastName
1	محمد	علیزاده

توضیح مثال: در رویه فوق ابتدا پارامتر @lastName را بررسی می کند اگر NULL باشد مقدار "کبیری" را بجای این پارامتر قرار می دهد، پس از آن نام و نام خانودگی کارمندی را که نام خانوادگی آن شبیه پارامتر ورودی می باشد را بازگشت می دهد. همانطور که دیده می شود دو بار تابع را اجرا کردیم، یکبار بدون پارامتر که بجای آن مقدار پیش فرض NULL قرار می گیرد و بنابراین در بدنه برنامه بجای پارامتر @lastName مقدار "کبیری" قرار می گیرد و سطر مربوط به آن برگشت داده می شود. ولی در فراخوانی بعدی پارامتر دارای مقدار می باشد، و نام و نام خانوادگی مربوط به همان پارامتر برگشت داده شده است.

نکته: رویه ها می توانند مانند توابع مقادیر مورد نظر را به محل فراخوانی بازگشت دهند، به این منظور می توان از مفهوم پارامتر خروجی استفاده نمود، در صورتیکه در توابع پارامترها فقط از نوع ورودی بودند.

مثال ۸-۸: رویه ای بنویسید که محیط و مساحت یک مستطیل را بازگشت دهد.

```

Create procedure area_perim (@area real output , @perimeter real output, @width real=10,@height real=20)
As
Begin
    Set @area =@width *@height
    Set @perimeter=2*(@width + @height)
end

```

100 %

Messages

Command(s) completed successfully.

اجرای این رویه در شکل زیر آورده شده است.

```

Declare @a real , @p real
Execute area_perim @a output,@p output,2,6
print 'area is: '
Print @a
print 'perimeter is: '
Print @p

```

100 %

Messages

area is:
12
perimeter is:
16

توضیح مثال: در این مثال شما باید مقادیری را صراحتاً بازگشت دهید(توجه کنید مانند مثال قبل خروجی از جدول SELECT نمی شود) بنابراین نیاز به تعریف پارامتر خروجی نیز دارید. همان طور که مشاهده می کنید برای تعریف پارامتر خروجی کلمه **output** را پس از نوع داده ای و تعریف رویه آورده ایم، و هنگام اجرا نیز بعد از نام متغیر تعریف شده به منظور قرار گرفتن خروجی در آن کلمه **Output** را آورده ایم.

نکته: رویه ها را میتوان در زمان ایجاد مستقیماً کامپایل نمود به این منظور میتوان گزینه WITH RECOMPILE را بلافاصله بعد از CREATE PROCEDURE آورد. در این حالت رویه بلافاصله پس از اجرا کامپایل میشود.

مثال ۸-۹: رویه ای بنویسید که مساحت یک مستطیل را برگرداند. (با RECOMPILE)

```
CREATE PROCEDURE area (@varx int = 2 , @vary int = 3)
WITH RECOMPILE
AS
BEGIN
PRINT @varx * @vary
End
```

Messages
Command(s) completed successfully.

در این صورت فقط بار اول که رویه اجرا میشود کامپایل میشود (روی حافظه، نه دیسک)، ولی در بقیه موارد رویه فقط اجرا میشود بدون کامپایل.

۸-۴ تغییر توابع و رویه ها: به منظور تغییر بدنه توابع و رویه ها کافیست بجای "CREATE PROCEDURE /FUNCTION" از "ALTER PROCEDURE /FUNCTION" استفاده کنید.

مثال ۸-۱۰: رویه ی مثال ۸-۶ را طوری تغییر دهید که نام درس را بعنوان ورودی دریافت کند.

```
ALTER PROCEDURE Course_Info (@CourseName As nvarchar(50))
AS BEGIN
SELECT * FROM Course WHERE CoTitle= @CourseName
END
```

Messages
Command(s) completed successfully.

۸-۵ حذف توابع و رویه ها: به منظور حذف توابع و رویه ها به سادگی از دستور DROP به صورت زیر استفاده می کنیم.

```
DROP { PROC | PROCEDURE } { procedure } [ , ...n ]
```

```
DROP FUNCTION { [ schema_name. ] function_name } [ ,...n ]
```

همانگونه که مشاهده می کنید می توان از کلمه کلیدی **procedure** یا **proc** برای رویه ها و **function** برای توابع استفاده کرد، بعد از آن نام یک رویه (تابع) یا چندین رویه (یا تابع که روی یک دیتابیس هستند و با کاما از هم جدا شده اند) را بیاورید.

مثال ۸- ۱۱: دستوری بنویسید که رویه های **listOfEmployee** و **area** را که در مثال های قبل بوده اند (روی یک دیتابیس) حذف کند.



۸- ۶ کنترل جریان

زمانی که با رویه ذخیره شده کار می کنیم گاهی لازم است جریان اطلاعات را در آن کنترل نماییم. کنترل جریان را می توانید با عبارات شرطی مانند **IF.....ELSE** یا حلقه های تکرار مانند **WHILE.....BREAK** انجام دهید.

عبارت IF.....ELSE

تا کنون تمام دستورات در هر شرایطی اجرا می شدند. اما گاهی لازم است بر اساس حاصل یک عبارت منطقی (**TRUE** یا **FALSE**) دستوری را اجرا کنیم یا اجرا نکنیم. فرمت دستور به صورت زیر می باشد:

```
IF Boolean_expression\
    sql_statement\
[ ELSE IF Boolean_expression\
    sql_statement\
.
.
.
ELSE
sql_statement_n]
```

در مقابل IF یک عبارت بولین یا دستور SELECT می تواند قرار بگیرد، اگر حاصل ارزیابی Boolean_expression_۱ برابر با TRUE باشد sql_statement_۱ که دستورات SQL است اجرا می شود و پس از آن دستوری دیگر اجرا نمی شود، اما اگر نتیجه FALSE باشد اجرا نشده و سراغ ارزیابی Boolean_expression_۲ می رود، اگر حاصل TRUE باشد sql_statement_۲ اجرا می شود و و پس از آن دستوری دیگر اجرا نمی شود ولی اگر FALSE باشد سراغ sql_statement_۳ می رود، و به این ترتیب تا انتها... اگر هیچ یک از Boolean_expression ها حاصل TRUE نداشته باشد دستورات مقابل ELSE یعنی sql_statement_n اجرا می شود.

مثال: دستوری بنویسید که متغیری را تعیین علامت کند.

```

DECLARE @num AS int
SET @num=-12

IF @num>0
PRINT 'POSITIVE'
ELSE IF @num<0
PRINT 'NEGATIVE'
ELSE
PRINT 'ZERO'

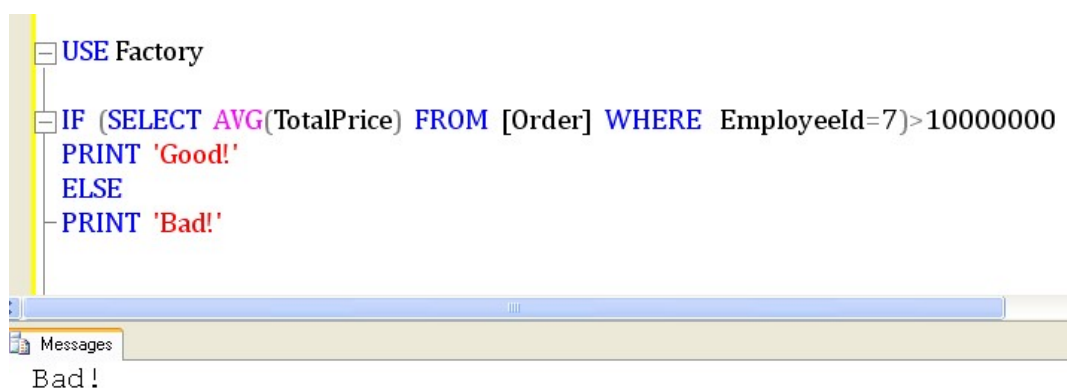
```

Messages

NEGATIVE

توضیح مثال: ابتدا یک متغیر به نام num تعریف شده و مقدار ۱۲- گرفته ، پس از آن در اولین IF بررسی شده است که آیا num بزرگتر از صفر است یا نه، چون ۱۲- از صفر بزرگتر نیست حاصل بررسی اولین عبارت FALSE می شود، بنابراین سراغ ELSE IF می رود و عبارت مقابل آن را بررسی می کند، چون num از صفر کوچک تر است، حاصل این ارزیابی TRUE می شود و دستور مقابل آن یعنی چاپ NEGATIVE اجرا می شود.

مثال: دستوری بنویسید که اگر میانگین فروش کارمند شماره ۷ بیش از ۱۰۰۰۰۰۰۰ باشد پیغام good و در غیر اینصورت پیغام bad صادر کند.



توضیح مثال: در این مثال دستور SELECT یک عدد که میانگین فروش کارمند شماره ۷ می باشد را برگشت داده است، که این عدد با ۱۰۰۰۰۰۰۰ مقایسه می شود، اگر بزرگتر باشد حاصل TRUE برگشت داده می شود و عبارت Good چاپ می شود در غیر این صورت FALSE شده و Bad را چاپ می کند. توجه کنید چون ما میانگین فروش را فقط برای یک کارمند خاص می خواهیم از تابع جمعی (avg) بدون GROUP BY استفاده کردیم.

نکته : اگر دستوراتی که بعد از بررسی شرط باید اجرا شوند بیش از یک دستور باشد، باید از BEGIN و END در ابتدا و انتهای دستورات استفاده کنید.

عبارت WHILE...BREAK

با استفاده از این عبارت می توانید یک یا چندین دستور را بارها پشت سر هم اجرا کنید، در واقع یک عبارت شرطی درون WHILE دائماً بررسی می شود تا هنگامی که این عبارت TRUE باشد دستورات اجرا می شوند و به محض FALSE شدن آن شرط(ها) دیگر دستورات اجرا نمی شوند و با کلید واژه BREAK خروج از حلقه صورت می گیرد. فرمت WHILE به شکل زیر می باشد:

```
WHILE Boolean_expression
{ sql_statement | BREAK | CONTINUE }
```

در فرمت فوق کلید واژه CONTINUE پردازش را به ابتدای WHILE برده و دستورات پس از خود را اجرا نمی کند. همانند دستورات شرطی اگر دستورات از یک دستور بیش تر باشد از BEGIN و END استفاده می کنیم.

مثال: خروجی دستورات زیر چیست؟

```

DECLARE @LoopCount AS int,@TestCount AS int
SET @LoopCount=0
SET @TestCount=0
WHILE @LoopCount<20
BEGIN
    SET @LoopCount=@LoopCount+1
    SET @TestCount=@TestCount+1
    SELECT @LoopCount AS LoopCount ,@TestCount AS TestCount
    IF @TestCount>10
        BREAK
    ELSE
        CONTINUE
    SELECT @LoopCount AS LoopCount ,@TestCount AS TestCount
END

```

خروجی به شکل زیر حاصل می شود:

Results		Messages
LoopCount	TestCount	
1	1	
1	2	
1	3	
1	4	
1	5	
1	6	
1	7	
1	8	
1	9	
1	10	
1	11	

توضیح مثال: ابتدا دو متغیر با مقادیر اولیه صفر تعریف می شوند. سپس در حلقه WHILE تا هنگامیکه @LoopCount کوچکتر از ۲۰ است باید مقادیر دو متغیر چاپ شوند، هر دو متغیر هم در هر دور حلقه یکی افزایش می یابد، بنابراین باید ۲۰ بار حلقه تکرار شود، اما شرطی در حلقه قرار دارد که چک می کند اگر متغیر @TestCount مقدارش از ۱۰ بیشتر باشد کلاً از حلقه خارج

شود (دستور BREAK). بنابراین هنگامی که دو متغیر مقدار ۱۱ می گیرند دیگر دستورات اجرا نمی شوند. توجه کنید که دستور SELECT بعد از CONTINUE هیچ گاه اجرا نمی شود.

مثال: فرض کنید در جدول TEACHER تا شماره استاد ۱۰ را در جدول داریم، میخواهیم تا شماره استادی ۲۰ را به جدول بدون مقدار دهی سایر فیلدها اضافه کنیم. با استفاده از حلقه این کار را انجام دهید.

```

DECLARE @INC AS int
SET @INC=11
WHILE @INC<21
BEGIN
    SELECT CAST(@INC AS CHAR(2))
    INSERT INTO Teacher (TeachId,TeachName) VALUES (@INC,N'بی نام')
    SELECT CAST(@INC AS INT)
    SET @INC=@INC+1
END

```

توضیح مثال: ابتدا یک متغیر به نام Inc با مقدار اولیه ۱۱ تعریف شده است، چون این متغیر باید یک واحد یک واحد افزایش یابد از نوع INT تعریف می شود. از طرفی چون TeachId از نوع کاراکتری است باید به نوع کاراکتری تبدیل شود. پس از اجرای دستورات فوق در جدول استاد شماره استادی یعنی TeachId مقادیر ۱۱ تا ۲۰ و نام استاد یعنی TeachName مقدار "بی نام" را می گیرند سپس دوباره متغیر Inc به نوع داده INT تبدیل میشود. پس از آن می توانید مقدار فیلدها را تغییر دهید. اگر از حلقه استفاده نمی کردید باید ۱۰ بار مقادیر شماره استادی را مقدار دهی می کردید.

نکته: در عمل برای این گونه ستون ها که مقادیر ترتیبی پشت سر هم میگیرند خصوصیت Identity آن را مقدار دهی میکند تا خود SQL Server به صورت خودکار مقدار دهی ستون را انجام دهد.